

Preface

In a rapidly evolving world, where change is the only constant, we find ourselves intertwined with the marvels of technology. The advent of mobile devices and the internet has seamlessly integrated into every aspect of our lives, becoming an inseparable part of our daily existence. Moreover, the automotive industry is witnessing a remarkable transformation, as we shift from traditional vehicles to the era of autonomous cars, a realm where machines take the wheel, revolutionising transportation as we know it.

As we navigate through this futuristic landscape, we are accompanied by a plethora of helpful bots that cater to our every need, from booking tickets for our adventures to arranging all our travel necessities. Even the financial realm is not untouched, with banks now being supported by these intelligent bots, ensuring seamless transactions and personalised services for every customer.

Underlying this remarkable transformation lies the fascinating domain of artificial intelligence and data science, two interwoven disciplines that orchestrate this magic. And at the very core of this enchanting world is Python, a programming language that effortlessly weaves its spell, making the seemingly complex concepts appear astonishingly simple.

The desire to impart this knowledge and power to the younger generation has led me to envision a future where my students can not only grasp the intricacies of Python but also excel in harnessing its potential. To sow the seeds of this digital journey, the foundation of Python can be expertly cultivated within the walls of schools, empowering the minds of our students with a valuable skill set that will serve them well in their journey through life.

This endeavour represents the first step in this direction, a humble attempt to introduce the wonders of Python to those eager minds. I am immensely grateful for the unwavering support of my beloved wife and children, who have actively contributed to creating enriching content for the students. I also extend my heartfelt thanks to the principals and heads of educational institutions who have enthusiastically embraced this initiative and incorporated it into their school curriculum, fostering an environment of growth and exploration.

With this book, my earnest hope is to lay the groundwork for your learning journey, enabling you to grasp the fundamentals of Python with ease and confidence. Embrace the knowledge, and may it empower you to embark on your own quests of innovation and discovery. May the blessings of the divine guide you every step of the way. God bless you all!

Mohammed Arif Shaikh

Introducing My Team	
Mrs. Ghazala Shaikh - M.A. English, Project Manager: Leading the Python Introduction Project in Schools. Miss. Zainab Shaikh - Pursuing Dip. Computer Engg (MH Saboo Siddik): Crafting engaging content and informative videos for better comprehension. Mr. Yusuf Shaikh - Pursuing Dip. Computer Engg (MH Saboo Siddik): Providing technical expertise and conducting material reviews. Together, we are committed to delivering excellence in our Python Introduction initiative.	

Simple Program



<https://youtu.be/SDO9Q9ljgAI>

```
print("Twinkle Twinkle little star")
print("How I wonder what you are")
print("Up above the world so high")
print("Like a diamond in the sky")
```

The `print()` function in Python is used to display or print something on the screen.

In this line, we are using the `print()` function to show the text "Twinkle Twinkle little star" on the screen. When the code runs, you will see this line printed out, just like when you see words on a page

When you put all the lines of code together and run the program, the computer will show the lyrics of the song "Twinkle, Twinkle, Little Star" on the screen, one line after the other. It's like the computer is singing the song for you by showing you the words!

You can think of this code as a set of instructions for the computer to display the song's lyrics, just like how you follow instructions



<https://youtu.be/SDO9Q9ljgAI>

```
print("""Twinkle Twinkle little star
How are wonder what you are
Up above the world so high
Like a diamond in the sky""")
```

These triple quotes `"""` are used to create a multi-line string in Python.

With triple quotes, you can write multiple lines of text as a single string, preserving the line breaks.

Between the triple quotes, we have the lyrics to the song "Twinkle, Twinkle, Little Star." Each line represents one line of the song.

You might notice that the second, third, and fourth lines of the lyrics have an extra space at the beginning (four spaces). This is called indentation. It's used here to create a specific visual formatting, making the lines slightly indented compared to the first line.

	When the code is executed, the print() function will display the lyrics on the screen exactly as they are written within the triple quotes. The first line will be displayed as is, and the subsequent lines will have an extra space at the beginning due to the indentation.
--	---

DataTypes

 https://youtu.be/SDO9Q9ljgAI	"Integer": This is a string literal. It is a sequence of characters enclosed in double quotes. In this case, the string is "Integer." This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement. 25: This is an integer literal. It represents a whole number without any decimal points. In this case, the integer is 25. When the code is executed, the print() function will display the string "Integer" followed by the integer 25, separated by a space. So, the output of this code will be "Integer 25" on the screen. It's as if the computer is saying, "Hey, I have two things to show you: the word 'Integer' and the number 25!" And it shows them together with a space in between.
---	---

 https://youtu.be/SDO9Q9ljgAI	"Float": This is a string literal, just like in the previous example. It is a sequence of characters enclosed in double quotes. In this case, the string is "Float." This comma is used to separate the items that need to be printed. Like before, it
---	--

	allows us to print multiple things in the same <code>print()</code> statement. 25.5: This is a floating-point literal. It represents a decimal number. In this case, the float is 25.5. When the code is executed, the <code>print()</code> function will display the string "Float" followed by the floating-point number 25.5, separated by a space. So, the output of this code will be "Float 25.5" on the screen. It's like the computer is telling us, "Hey, I have two things to show you: the word 'Float' and the number 25.5!" And it displays them together with a space in between.
--	--

 https://youtu.be/SDO9Q9ljgAI	"String": This is a string literal, just like in the previous examples. It is a sequence of characters enclosed in double quotes. In this case, the string is "String." This comma is used to separate the items that need to be printed. As before, it allows us to print multiple things in the same <code>print()</code> statement. "Hello": This is another string literal. It represents a different sequence of characters enclosed in double quotes. In this case, the string is "Hello." When the code is executed, the <code>print()</code> function will display the string "String" followed by the string "Hello," separated by a space. So, the output of this code will be "String Hello" on the screen. It's like the computer is saying, "Hey, I have two things to show you: the word 'String' and the word 'Hello'!" And it displays them together with a space in between.
--	--

Conversion between Data Types

 https://youtu.be/SDO9Q9ljgAI <pre>print (float(5))</pre>	<code>float()</code>: This is a built-in Python function used to convert a value to a floating-point number. 5: This is an integer literal. It represents the whole number 5. When the code is executed, the <code>float()</code> function will convert the integer 5 into a floating-point number, which means a number with decimal points. Since 5 can be represented as 5.0 in floating-point format, the <code>print()</code> function will display the result. So, the output of this code will be 5.0 on the screen. It's like the computer is saying, "Hey, you gave me the number 5, and I turned it into 5.0 for you!"
 https://youtu.be/SDO9Q9ljgAI <pre>print(int(10.6))</pre>	<code>int()</code>: This is a built-in Python function used to convert a value to an integer. 10.6: This is a floating-point literal. It represents a decimal number. When the code is executed, the <code>int()</code> function will convert the floating-point number 10.6 into an integer. When converting from a float to an integer, Python truncates (removes) the decimal part and keeps only the whole number part. So, the <code>print()</code> function will display the result. The output of this code will be 10 on the screen. It's like the computer is saying, "Hey, you gave me the number 10.6, and I turned it into the whole number 10 for you!"
 https://youtu.be/SDO9Q9ljgAI <pre>print(str(35.5))</pre>	<code>str()</code>: This is a built-in Python function used to convert a value into a string.

	35.5: This is a floating-point literal. It represents a decimal number. When the code is executed, the <code>str()</code> function will convert the floating-point number 35.5 into a string. This means it will be represented as characters rather than as a numeric value. The <code>print()</code> function will display the result. As you can see, the number 35.5 has been converted to a string and enclosed in single quotes. The single quotes indicate that it is now a string value. So, the output of this code will be the string '35.5' on the screen. It's like the computer is saying, "Hey, you gave me the number 35.5, and I turned it into the characters '35.5' for you!"
--	---

Arithmetic Operator

 https://youtu.be/4pQWIEerz3A	<code>x = 15</code>: This line assigns the value 15 to the variable <code>x</code>. In Python, variables are like containers that can hold different values. Here, <code>x</code> will now hold the value 15. <code>y = 4</code>: This line assigns the value 4 to the variable <code>y</code>. Similar to the previous line, <code>y</code> will now hold the value 4.
---	---

 https://youtu.be/4pQWIEerz3A	<code>print()</code>: As we've seen before, <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>'x + y ='</code>: This is a string literal. It represents a sequence of characters enclosed in single quotes. In this case, the string is <code>'x + y ='</code>. <code>,:</code> This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement.
---	---

	$x + y$: This is an arithmetic expression that adds the values of variables x and y together. When the code is executed, the <code>print()</code> function will display the string '$x + y =$', followed by the result of the arithmetic expression $x + y$. So, the output of this code will be '$x + y = 19$' on the screen. It's like the computer is saying, "Hey, the value of x is 15, the value of y is 4, and when we add them together, we get 19!"
--	---

 https://youtu.be/4pQWIEerz3A	<code>print()</code> is a built-in Python function used to display or print something on the screen. '$x - y =$': This is a string literal, just like in the previous example. It represents a sequence of characters enclosed in single quotes. In this case, the string is '$x - y =$'. ,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement. $x - y$: This is an arithmetic expression that subtracts the value of variable y from the value of variable x. When the code is executed, the <code>print()</code> function will display the string '$x - y =$', followed by the result of the arithmetic expression $x - y$. So, the output of this code will be '$x - y = 11$' on the screen. It's like the computer is saying, "Hey, the value of x is 15, the value of y is 4, and when we subtract y from x, we get 11!"
---	--



<https://youtu.be/4pQWIEerz3A>

```
print('x * y =', x * y)
```

print(): print() is a built-in Python function used to display or print something on the screen.

'x * y =': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'x * y ='.

,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement.

x * y: This is an arithmetic expression that multiplies the values of variables x and y.

When the code is executed, the print() function will display the string 'x * y =', followed by the result of the arithmetic expression x * y.

So, the output of this code will be 'x * y = 60' on the screen. It's like the computer is saying, "Hey, the value of x is 15, the value of y is 4, and when we multiply them together, we get 60!"



<https://youtu.be/4pQWIEerz3A>

```
print('x / y =', x / y)
```

print(): print() is a built-in Python function used to display or print something on the screen.

'x / y =': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'x / y ='.

,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement.

x / y: This is an arithmetic expression that divides the value of variable x by the value of variable y.

	When the code is executed, the <code>print()</code> function will display the string <code>'x / y ='</code>, followed by the result of the arithmetic expression <code>x / y</code>. So, the output of this code will be <code>'x / y = 3.75'</code> on the screen. It's like the computer is saying, "Hey, the value of x is 15, the value of y is 4, and when we divide x by y, we get 3.75!"
--	---

 https://youtu.be/4pQWIEerz3A	<code>print('x // y =', x // y)</code>	<code>print()</code>: <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>'x // y ='</code>: This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is <code>'x // y ='</code>. <code>,:</code> This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement. <code>x // y</code>: This is an arithmetic expression that performs integer division. It divides the value of variable <code>x</code> by the value of variable <code>y</code>, but the result is a whole number without any decimal points. When the code is executed, the <code>print()</code> function will display the string <code>'x // y ='</code>, followed by the result of the arithmetic expression <code>x // y</code>. So, the output of this code will be <code>'x // y = 3'</code> on the screen. It's like the computer is saying, "Hey, the value of x is 15, the value of y is 4, and when we divide x by y, we get 3 as the whole number result!"
---	---	--



<https://youtu.be/4pQWIEerz3A>

```
print('x % y = ', x % y)
```

print(): print() is a built-in Python function used to display or print something on the screen.

'x % y =': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'x % y ='.

,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement.

x % y: This is an arithmetic expression that calculates the remainder when x is divided by y. It's called the modulus operation.

When the code is executed, the print() function will display the string 'x % y =', followed by the result of the arithmetic expression x % y.

So, the output of this code will be 'x % y = 3' on the screen. It's like the computer is saying, "Hey, the value of x is 15, the value of y is 4, and when we divide x by y, we get a remainder of 3!"



<https://youtu.be/4pQWIEerz3A>

```
print('x ** y = ', x ** y)
```

print(): print() is a built-in Python function used to display or print something on the screen.

'x ** y =': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'x ** y ='.

,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement.

x ** y: This is an arithmetic expression that performs exponentiation. It calculates the value of x raised to the power of y.

	When the code is executed, the <code>print()</code> function will display the string <code>'x ** y ='</code>, followed by the result of the arithmetic expression <code>x ** y</code>. So, the output of this code will be <code>'x ** y = 50625'</code> on the screen. It's like the computer is saying, "Hey, the value of x is 15, and the value of y is 4. When we raise x to the power of y, we get 50625!"
--	--

Comparison Operator

 https://youtu.be/PY-IQs_dP5g	<code>x = 10</code>: This line assigns the value 10 to the variable <code>x</code>. As we discussed earlier, variables are like containers that can hold different values. Here, <code>x</code> will now hold the value 10. <code>y = 12</code>: This line assigns the value 12 to the variable <code>y</code>. Similar to the previous line, <code>y</code> will now hold the value 12.
---	--

 https://youtu.be/PY-IQs_dP5g	<code>print('x > y is ', x > y)</code> <code>print()</code>: <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>'x > y is'</code>: This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is <code>'x > y is'</code>. <code>,:</code> This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement. <code>x > y</code>: This is a comparison expression. It checks if the value of <code>x</code> is greater than the value of <code>y</code>. The result will be either <code>True</code> or <code>False</code>. When the code is executed, the <code>print()</code> function will display the string <code>'x > y is'</code>,
---	---

	followed by the result of the comparison expression <code>x > y</code>. So, the output of this code will be <code>'x > y is False'</code> on the screen. It's like the computer is saying, "Hey, the value of <code>x</code> is 10, the value of <code>y</code> is 12, and when we check if <code>x</code> is greater than <code>y</code>, the answer is False!"
--	--

 https://youtu.be/PY-IQs_dP5g	<code>print()</code>: <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>'x < y is'</code>: This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is <code>'x < y is'</code>. <code>,:</code> This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement. <code>x < y</code>: This is a comparison expression. It checks if the value of <code>x</code> is less than the value of <code>y</code>. The result will be either <code>True</code> or <code>False</code>. When the code is executed, the <code>print()</code> function will display the string <code>'x < y is'</code>, followed by the result of the comparison expression <code>x < y</code>. So, the output of this code will be <code>'x < y is True'</code> on the screen. It's like the computer is saying, "Hey, the value of <code>x</code> is 10, the value of <code>y</code> is 12, and when we check if <code>x</code> is less than <code>y</code>, the answer is True!"
---	---

 https://youtu.be/PY-IQs_dP5g	<code>print()</code>: <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>'x == y is'</code>: This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is <code>'x == y is'</code>.
---	--

	,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement. x == y: This is a comparison expression using the double equals == operator. It checks if the value of x is equal to the value of y. The result will be either True or False. When the code is executed, the print() function will display the string 'x == y is', followed by the result of the comparison expression x == y. So, the output of this code will be 'x == y is False' on the screen. It's like the computer is saying, "Hey, the value of x is 10, the value of y is 12, and when we check if x is equal to y, the answer is False!"
--	---

 https://youtu.be/PY-IQs_dP5g <pre>print('x != y is ', x != y)</pre>	print(): print() is a built-in Python function used to display or print something on the screen. 'x != y is': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'x != y is'. ,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement. x != y: This is a comparison expression using the != operator. It checks if the value of x is not equal to the value of y. The result will be either True or False. When the code is executed, the print() function will display the string 'x != y is', followed by the result of the comparison expression x != y. So, the output of this code will be 'x != y is True' on the screen. It's like the computer is
---	---

	saying, "Hey, the value of x is 10, the value of y is 12, and when we check if x is not equal to y, the answer is True!"
--	--

 https://youtu.be/PY-IQs_dP5g <pre>print('x >= y is ', x >= y)</pre>	print(): print() is a built-in Python function used to display or print something on the screen. 'x >= y is': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'x >= y is'. ,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement. x >= y: This is a comparison expression using the >= operator. It checks if the value of x is greater than or equal to the value of y. The result will be either True or False. When the code is executed, the print() function will display the string 'x >= y is', followed by the result of the comparison expression x >= y. So, the output of this code will be 'x >= y is False' on the screen. It's like the computer is saying, "Hey, the value of x is 10, the value of y is 12, and when we check if x is greater than or equal to y, the answer is False!"
---	---

 https://youtu.be/PY-IQs_dP5g <pre>print('x <= y is ', x <= y)</pre>	print(): print() is a built-in Python function used to display or print something on the screen. 'x <= y is': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'x <= y is'. ,: This comma is used to separate the items that need to be printed. It allows us to print
---	--

	multiple things in the same print() statement. <code>x <= y</code>: This is a comparison expression using the <code><=</code> operator. It checks if the value of <code>x</code> is less than or equal to the value of <code>y</code>. The result will be either <code>True</code> or <code>False</code>. When the code is executed, the <code>print()</code> function will display the string <code>'x <= y is'</code>, followed by the result of the comparison expression <code>x <= y</code>. So, the output of this code will be <code>'x <= y is True'</code> on the screen. It's like the computer is saying, "Hey, the value of <code>x</code> is 10, the value of <code>y</code> is 12, and when we check if <code>x</code> is less than or equal to <code>y</code>, the answer is <code>True</code>!"
--	--

Logical Operator

 https://youtu.be/yu68e8omt4k	<code>isTRUE = True</code>: This line assigns the boolean value <code>True</code> to the variable <code>isTRUE</code>. Boolean values represent the concept of truth and can only have two possible values: <code>True</code> or <code>False</code>. <code>isFALSE = False</code>: This line assigns the boolean value <code>False</code> to the variable <code>isFALSE</code>.
---	---

 https://youtu.be/yu68e8omt4k	<code>print()</code>: <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>'isTRUE and isTRUE ==> '</code>: This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is <code>'isTRUE and isTRUE ==> '</code>. <code>,</code>: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement.
---	---

	isTRUE and isTRUE: This is a logical expression using the and operator. It checks if both isTRUE variables are True. The result will be either True or False. When the code is executed, the print() function will display the string 'isTRUE and isTRUE ==>', followed by the result of the logical expression isTRUE and isTRUE. So, the output of this code will be 'isTRUE and isTRUE ==> True' on the screen. It's like the computer is saying, "Hey, both isTRUE variables are True, so the result of isTRUE and isTRUE is True!"
--	---

 https://youtu.be/yu68e8omt4k <pre>print('isTRUE and isFALSE ==> ', isTRUE and isFALSE)</pre>	print(): print() is a built-in Python function used to display or print something on the screen. 'isTRUE and isFALSE ==>': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'isTRUE and isFALSE ==>'. ,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement. isTRUE and isFALSE: This is a logical expression using the and operator. It checks if both isTRUE and isFALSE variables are True. However, since isFALSE has the value False, the entire expression will be evaluated to False. When the code is executed, the print() function will display the string 'isTRUE and isFALSE ==>', followed by the result of the logical expression isTRUE and isFALSE. So, the output of this code will be 'isTRUE and isFALSE ==> False' on the screen. It's like the computer is saying, "Hey, isTRUE is
--	--

	True, but isFALSE is False, so the result of isTRUE and isFALSE is False!"
--	--

 https://youtu.be/yu68e8omt4k <pre>print('isFALSE and isTRUE => ', isFALSE and isTRUE)</pre>	print(): print() is a built-in Python function used to display or print something on the screen. 'isFALSE and isTRUE =>': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'isFALSE and isTRUE =>'. ,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement. isFALSE and isTRUE: This is a logical expression using the and operator. It checks if both isFALSE and isTRUE variables are True. However, since isFALSE has the value False, the entire expression will be evaluated to False. When the code is executed, the print() function will display the string 'isFALSE and isTRUE =>', followed by the result of the logical expression isFALSE and isTRUE. So, the output of this code will be 'isFALSE and isTRUE => False' on the screen. It's like the computer is saying, "Hey, isFALSE is False, and since and requires both expressions to be True, the result of isFALSE and isTRUE is False!"
---	--

 https://youtu.be/yu68e8omt4k <pre>print('isFALSE and isFALSE => ', isFALSE and isFALSE)</pre>	print(): print() is a built-in Python function used to display or print something on the screen. 'isFALSE and isFALSE =>': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'isFALSE and isFALSE =>'.
---	--

	,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement. isFALSE and isFALSE: This is a logical expression using the and operator. It checks if both isFALSE variables are True. However, since both isFALSE variables have the value False, the entire expression will be evaluated to False. When the code is executed, the print() function will display the string 'isFALSE and isFALSE ==>', followed by the result of the logical expression isFALSE and isFALSE. So, the output of this code will be 'isFALSE and isFALSE ==> False' on the screen. It's like the computer is saying, "Hey, both isFALSE variables are False, so the result of isFALSE and isFALSE is False!"
--	--

 https://youtu.be/yu68e8omt4k <pre>print('isTRUE or isTRUE ==> ', isTRUE or isTRUE)</pre>	print(): print() is a built-in Python function used to display or print something on the screen. 'isTRUE or isTRUE ==>': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'isTRUE or isTRUE ==>'. ,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement. isTRUE or isTRUE: This is a logical expression using the or operator. It checks if either of the isTRUE variables is True. Since both isTRUE variables have the value True, the entire expression will be evaluated to True.
--	--

	When the code is executed, the <code>print()</code> function will display the string 'isTRUE or isTRUE ==>', followed by the result of the logical expression <code>isTRUE or isTRUE</code>. So, the output of this code will be 'isTRUE or isTRUE ==> True' on the screen. It's like the computer is saying, "Hey, at least one of the <code>isTRUE</code> variables is True, so the result of <code>isTRUE or isTRUE</code> is True!"
--	---

 https://youtu.be/yu68e8omt4k	<code>print()</code>: <code>print()</code> is a built-in Python function used to display or print something on the screen. 'isTRUE or isFALSE ==>': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'isTRUE or isFALSE ==>'. ,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement. <code>isTRUE or isFALSE</code>: This is a logical expression using the <code>or</code> operator. It checks if either of the <code>isTRUE</code> or <code>isFALSE</code> variables is True. Since <code>isTRUE</code> has the value True, the entire expression will be evaluated to True. When the code is executed, the <code>print()</code> function will display the string 'isTRUE or isFALSE ==>', followed by the result of the logical expression <code>isTRUE or isFALSE</code>. So, the output of this code will be 'isTRUE or isFALSE ==> True' on the screen. It's like the computer is saying, "Hey, at least one of the variables is True, so the result of <code>isTRUE or isFALSE</code> is True!"
---	---

 https://youtu.be/yu68e8omt4k	<code>print()</code>: <code>print()</code> is a built-in Python function used to display or print something on the screen.
---	---

```
print('isFALSE or isTRUE => ',  
      isFALSE or isTRUE)
```

'isFALSE or isTRUE =>': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'isFALSE or isTRUE =>'.

,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement.

isFALSE or isTRUE: This is a logical expression using the or operator. It checks if either of the isFALSE or isTRUE variables is True. Since isTRUE has the value True, the entire expression will be evaluated to True.

When the code is executed, the print() function will display the string 'isFALSE or isTRUE =>', followed by the result of the logical expression isFALSE or isTRUE. So, the output of this code will be 'isFALSE or isTRUE => True' on the screen. It's like the computer is saying, "Hey, at least one of the variables is True, so the result of isFALSE or isTRUE is True!"



<https://youtu.be/yu68e8omt4k>

```
print('isFALSE or isFALSE => ',  
      isFALSE or isFALSE)
```

print(): print() is a built-in Python function used to display or print something on the screen.

'isFALSE or isFALSE =>': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'isFALSE or isFALSE =>'.

,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement.

isFALSE or isFALSE: This is a logical expression using the or operator. It checks if either of the isFALSE variables is True.

	Since both <code>isFALSE</code> variables have the value <code>False</code>, the entire expression will be evaluated to <code>False</code>. When the code is executed, the <code>print()</code> function will display the string <code>'isFALSE or isFALSE ==>'</code>, followed by the result of the logical expression <code>isFALSE or isFALSE</code>. So, the output of this code will be <code>'isFALSE or isFALSE ==> False'</code> on the screen. It's like the computer is saying, "Hey, both <code>isFALSE</code> variables are <code>False</code>, so the result of <code>isFALSE or isFALSE</code> is <code>False</code>!"
--	--

 https://youtu.be/yu68e8omt4k	<code>print('not isTRUE ==> ', not isTRUE)</code>	<code>print()</code>: <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>'not isTRUE ==>'</code>: This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is <code>'not isTRUE ==>'</code>. <code>,</code>: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement. <code>not isTRUE</code>: This is a logical expression using the <code>not</code> operator. It negates the value of <code>isTRUE</code>, which means if <code>isTRUE</code> is <code>True</code>, the result will be <code>False</code>, and if <code>isTRUE</code> is <code>False</code>, the result will be <code>True</code>. When the code is executed, the <code>print()</code> function will display the string <code>'not isTRUE ==>'</code>, followed by the result of the logical expression <code>not isTRUE</code>. So, the output of this code will be <code>'not isTRUE ==> False'</code> on the screen. It's like the computer is saying, "Hey, <code>isTRUE</code> is <code>True</code>, but with <code>not</code>, the result is <code>False</code>!"
---	---	---



<https://youtu.be/yu68e8omt4>

```
print('not isFALSE => ', not isFALSE)
```

print(): print() is a built-in Python function used to display or print something on the screen.

'not isFALSE =>': This is a string literal, just like in the previous examples. It represents a sequence of characters enclosed in single quotes. In this case, the string is 'not isFALSE =>'.

,: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same print() statement.

not isFALSE: This is a logical expression using the not operator. It negates the value of isFALSE, which means if isFALSE is True, the result will be False, and if isFALSE is False, the result will be True.

When the code is executed, the print() function will display the string 'not isFALSE =>', followed by the result of the logical expression not isFALSE.

So, the output of this code will be 'not isFALSE => True' on the screen. It's like the computer is saying, "Hey, isFALSE is False, and with not, the result is True!"

Shortcut Operator



https://youtu.be/o7DPOO4_x54

```
x = 10  
y = 10
```

you have assigned the value 10 to both variables x and y



https://youtu.be/o7DPOO4_x54

```
x = x + 2  
y += 2
```

x = x + 2: In this line, you are updating the value of the variable x. The expression x + 2 adds 2 to the current value of x, which is 10. The result of this addition is 12, and now x will hold the value 12.

```
print("x = ", x, " y = ", y)
```

`y += 2`: This is a shorthand notation for updating the value of `y`. It is equivalent to `y = y + 2`. It adds 2 to the current value of `y`, which is also 10. The result of this addition is 12, and now `y` will hold the value 12 as well.

`print()`: As we've seen before, `print()` is a built-in Python function used to display or print something on the screen.

`"x =", "y ="`: These are string literals that represent the text `"x ="` and `"y ="`, respectively. They will be printed as part of the `print()` statement to label the output.

`,`: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same `print()` statement.

`x`: This is the variable `x` that holds the value 12 after the update.

`y`: This is the variable `y` that holds the value 12 after the update.

When the code is executed, the `print()` function will display the values of `x` and `y` with their respective labels.

So, the output of this code will be `x = 12 y = 12` on the screen, indicating that both `x` and `y` have been updated to 12 after performing the additions.



https://youtu.be/o7DPOO4_x54

```
x = x - 2
```

```
y -= 2
```

```
print("x = ", x, " y = ", y)
```

`x = x - 2`: In this line, you are updating the value of the variable `x`. The expression `x - 2` subtracts 2 from the current value of `x`, which is 12 (from the previous update). The result of this subtraction is 10, and now `x` will hold the value 10.

	<code>y -= 2</code>: This is a shorthand notation for updating the value of <code>y</code>. It is equivalent to <code>y = y - 2</code>. It subtracts 2 from the current value of <code>y</code>, which is also 12 (from the previous update). The result of this subtraction is 10, and now <code>y</code> will hold the value 10 as well. <code>print()</code>: As we've seen before, <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>"x =", "y ="</code>: These are string literals that represent the text <code>"x ="</code> and <code>"y ="</code>, respectively. They will be printed as part of the <code>print()</code> statement to label the output. <code>,</code>: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement. <code>x</code>: This is the variable <code>x</code> that now holds the value 10 after the subtraction. <code>y</code>: This is the variable <code>y</code> that now holds the value 10 after the subtraction. When the code is executed, the <code>print()</code> function will display the updated values of <code>x</code> and <code>y</code> with their respective labels. So, the output of this code will be <code>x = 10 y = 10</code> on the screen, indicating that both <code>x</code> and <code>y</code> have been updated to 10 after performing the subtractions.
--	--

 https://youtu.be/o7DPOO4_x54 <pre> x = x * 2 y *= 2 print("x = ", x, " y = ", y) </pre>	<code>x = x * 2</code>: In this line, you are updating the value of the variable <code>x</code>. The expression <code>x * 2</code> multiplies the current value of <code>x</code>, which is 10 (from the previous update), by 2. The result of this multiplication is 20, and now <code>x</code> will hold the value 20.
--	---

	<code>y *= 2</code>: This is a shorthand notation for updating the value of <code>y</code>. It is equivalent to <code>y = y * 2</code>. It multiplies the current value of <code>y</code>, which is also 10 (from the previous update), by 2. The result of this multiplication is 20, and now <code>y</code> will hold the value 20 as well. <code>print()</code>: As we've seen before, <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>"x =", "y ="</code>: These are string literals that represent the text <code>"x ="</code> and <code>"y ="</code>, respectively. They will be printed as part of the <code>print()</code> statement to label the output. <code>,</code>: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement. <code>x</code>: This is the variable <code>x</code> that now holds the value 20 after the multiplication. <code>y</code>: This is the variable <code>y</code> that now holds the value 20 after the multiplication. When the code is executed, the <code>print()</code> function will display the updated values of <code>x</code> and <code>y</code> with their respective labels. So, the output of this code will be <code>x = 20 y = 20</code> on the screen, indicating that both <code>x</code> and <code>y</code> have been updated to 20 after performing the multiplications.
--	--

 https://youtu.be/o7DPOO4_x54 <pre> x = x / 2 y /= 2 print("x = ", x, " y = ", y) </pre>	<code>x = x / 2</code>: In this line, you are updating the value of the variable <code>x</code>. The expression <code>x / 2</code> divides the current value of <code>x</code>, which is 20 (from the previous update), by 2. The result of this division is 10.0, and now <code>x</code> will hold the value 10.0. In Python, when you divide two integers, the result is automatically converted to a float.
--	---

	<code>y /= 2</code>: This is a shorthand notation for updating the value of <code>y</code>. It is equivalent to <code>y = y / 2</code>. It divides the current value of <code>y</code>, which is also 20 (from the previous update), by 2. The result of this division is 10.0, and now <code>y</code> will hold the value 10.0 as well. <code>print()</code>: As we've seen before, <code>print()</code> is a built-in Python function used to display or print something on the screen. <code>"x =", "y ="</code>: These are string literals that represent the text <code>"x ="</code> and <code>"y ="</code>, respectively. They will be printed as part of the <code>print()</code> statement to label the output. <code>,</code>: This comma is used to separate the items that need to be printed. It allows us to print multiple things in the same <code>print()</code> statement. <code>x</code>: This is the variable <code>x</code> that now holds the value 10.0 after the division. <code>y</code>: This is the variable <code>y</code> that now holds the value 10.0 after the division. When the code is executed, the <code>print()</code> function will display the updated values of <code>x</code> and <code>y</code> with their respective labels. So, the output of this code will be <code>x = 10.0 y = 10.0</code> on the screen, indicating that both <code>x</code> and <code>y</code> have been updated to 10.0 after performing the divisions.
--	---

Conditional Statement

 https://youtu.be/sBR0prl3jfo <pre>numStr = input('Enter the number: ') num = int(numStr)</pre>	<code>input()</code>: This is a built-in Python function used to take user input from the console. The function displays the message <code>'Enter the number: '</code> to prompt the user to enter a number.
---	---

```
if num > 0:  
    print(num, "is a positive number.")
```

numStr: This variable stores the input provided by the user as a string. The input is read as a string, even if the user enters a number, so it needs to be converted to an integer (using `int()`) for numerical comparisons.

`int()`: This is a built-in Python function used to convert a string to an integer. In this case, it converts the string numStr (the user input) to an integer, and the result is stored in the variable num.

if statement: This is a conditional statement in Python. It checks whether the condition inside the parentheses is True or False. If the condition is True, the indented block of code following the if statement will be executed.

`num > 0`: This is the condition being checked. It tests if the value of num is greater than 0.

`print()`: As we've seen before, `print()` is a built-in Python function used to display or print something on the screen.

num: This is the variable num, which contains the user's input converted to an integer.

"is a positive number.": This is a string literal that provides additional text to be displayed along with the value of num`.

If the user enters a positive number (greater than 0), the program will display a message indicating that the number is positive. Otherwise, if the user enters a non-positive number (0 or negative), nothing will be displayed because there is no corresponding else statement to handle those cases



<https://youtu.be/MKMKMia1aOU>

```
numStr = input('Enter the number: ')
num = int(numStr)
if num > 0:
    print(num, "is a positive number.")
else:
    print(num, "negative or zero")
```

input(): This is a built-in Python function used to take user input from the console. The function displays the message 'Enter the number: ' to prompt the user to enter a number.

numStr: This variable stores the input provided by the user as a string. The input is read as a string, even if the user enters a number, so it needs to be converted to an integer (using `int()`) for numerical comparisons.

int(): This is a built-in Python function used to convert a string to an integer. In this case, it converts the string `numStr` (the user input) to an integer, and the result is stored in the variable `num`.

if statement: This is a conditional statement in Python. It checks whether the condition inside the parentheses is True or False. If the condition is True, the indented block of code following the if statement will be executed.

num > 0: This is the condition being checked. It tests if the value of `num` is greater than 0.

print(): As we've seen before, `print()` is a built-in Python function used to display or print something on the screen.

else statement: This is an optional part of the conditional statement. If the condition specified in the if statement is False, the indented block of code following the else statement will be executed.

If the user enters a positive number (greater than 0), the program will display a message indicating that the number is positive. If the user enters a non-positive number (zero or negative), the program will display a message indicating that the number is negative or zero.



<https://youtu.be/MKMKMia1aOU>

```
numStr = input('Enter the number: ')
num = int(numStr)
if num > 0:
    print(num, "is a positive number.")
elif num < 0:
    print(num, "is a negative number.")
else:
    print(num, "is zero")
```

input(): This is a built-in Python function used to take user input from the console. The function displays the message 'Enter the number: ' to prompt the user to enter a number.

numStr: This variable stores the input provided by the user as a string. The input is read as a string, even if the user enters a number, so it needs to be converted to an integer (using `int()`) for numerical comparisons.

int(): This is a built-in Python function used to convert a string to an integer. In this case, it converts the string `numStr` (the user input) to an integer, and the result is stored in the variable `num`.

if statement: This is a conditional statement in Python. It checks whether the condition inside the parentheses is True or False. If the condition is True, the indented block of code following the if statement will be executed.

num > 0: This is the condition being checked. It tests if the value of `num` is greater than 0.

elif statement: This is short for "else if." It is an optional part of the conditional statement. If the previous if condition is False, the program will check the condition inside the elif statement. If that condition is True, the indented block of code following the elif statement will be executed.

num < 0: This is the condition being checked in the elif statement. It tests if the value of `num` is less than 0.

else statement: This is an optional part of the conditional statement. If both the previous if and elif conditions are False, the

	indented block of code following the else statement will be executed. If the user enters a positive number (greater than 0), the program will display a message indicating that the number is positive. If the user enters a negative number (less than 0), the program will display a message indicating that the number is negative. If the user enters 0, the program will display a message indicating that the number is zero.
--	---

 https://youtu.be/ZLQa9sGwc4A Find if the number is odd or even <pre>num = int(input("Enter number: ")) if num % 2 == 0: print("Number is even") else: print("Number is odd")</pre>	input(): This is a built-in Python function used to take user input from the console. The function displays the message "Enter number: " to prompt the user to enter a number. int(): This is a built-in Python function used to convert a string to an integer. The user's input, which is read as a string by input(), is converted to an integer and stored in the variable num. if statement: This is a conditional statement in Python. It checks whether the condition inside the parentheses is True or False. If the condition is True, the indented block of code following the if statement will be executed. num % 2 == 0: This is the condition being checked. It uses the modulo operator % to calculate the remainder of num divided by 2. If the remainder is 0, then the number is even. else statement: This is an optional part of the conditional statement. If the previous if condition is False, the indented block of code following the else statement will be executed. If the user enters a number, the program will check if it's even or odd. If the number is even (divisible by 2 without a remainder), it
---	--

	will display the message "Number is even." Otherwise, it will display "Number is odd."
--	---

 https://youtu.be/ZLQa9sGwc4	
Check if the number is positive, negative or zero <pre>num = int(input ("Enter the number: ")) if num > 0: print("Number is positive: ", num) elif num < 0: print("Number is Negative: ", num) else: print("Number is zero: ", num)</pre>	input(): This is a built-in Python function used to take user input from the console. The function displays the message "Enter the number: " to prompt the user to enter a number. int(): This is a built-in Python function used to convert a string to an integer. The user's input, which is read as a string by input(), is converted to an integer and stored in the variable num. if statement: This is a conditional statement in Python. It checks whether the condition inside the parentheses is True or False. If the condition is True, the indented block of code following the if statement will be executed. num > 0: This is the condition being checked. It tests if the value of num is greater than 0. elif statement: This is short for "else if." It is an optional part of the conditional statement. If the previous if condition is False, the program will check the condition inside the elif statement. If that condition is True, the indented block of code following the elif statement will be executed. num < 0: This is the condition being checked in the elif statement. It tests if the value of num is less than 0. else statement: This is an optional part of the conditional statement. If both the previous if and elif conditions are False, the indented block of code following the else statement will be executed. If the user enters a positive number (greater than 0), the program will display a message

	indicating that the number is positive. If the user enters a negative number (less than 0), the program will display a message indicating that the number is negative. If the user enters 0, the program will display a message indicating that the number is zero
--	--

Range

 https://youtu.be/mduKvQt_me4 <pre>a = [5, 10, 15, 20, 25, 30, 35, 40]</pre> 5 is at index 0 10 at index 1 and so on	<pre>a = [5, 10, 15, 20, 25, 30, 35, 40]</pre> A list is a data structure in Python that can hold multiple elements in a specific order. The elements in a list are separated by commas and are enclosed in square brackets. Each element in the list a represents a number: <pre>a[0]: 5, a[1]: 10, a[2]: 15, a[3]: 20, a[4]: 25, a[5]: 30, a[6]: 35, a[7]: 40,</pre> You can access individual elements in the list using their index. In Python, lists are zero-indexed, which means the first element has an index of 0, the second element has an index of 1, and so on. For example, if you want to access element 10, you can use <code>a[1]</code> because it's the second element in the list. Similarly, to access 30, you can use <code>a[5]</code>, and so on. Lists are a versatile data structure, and you can perform various operations on them, such as adding elements, removing elements, slicing, and iterating through the elements.
---	--

 https://youtu.be/mduKvQt_me4	The expression <code>a[2]</code> refers to the third element in the list a. Since Python uses zero-based indexing for lists, the first element has an index of 0,
---	--

<pre>print("a[2] = ", a[2])</pre>	the second element has an index of 1, and so on. Therefore, <code>a[2]</code> corresponds to the third element in the list <code>a</code>. Therefore, <code>a[2]</code> will be equal to 15. If you want to access and display the value at index 2 in the list <code>a</code>, you would write <code>a[2]</code>, which will give you the value 15.
-----------------------------------	---

 https://youtu.be/mduKvQt_me4 <pre>print("a[2:5] = ", a[2:5])</pre>	The slice notation <code>a[2:5]</code> extracts a portion of the list <code>a</code>, starting from index 2 (inclusive) up to, but not including, index 5 (exclusive). The slice <code>a[2:5]</code> will include elements from index 2 up to, but not including, index 5. So, it includes elements at index 2, 3, and 4. The result of <code>a[2:5]</code> will be a new list containing <code>[15, 20, 25]</code>. This is because it includes the elements 15, 20, and 25 from the original list <code>a</code>. Therefore, the slice <code>a[2:5]</code> gives us the list <code>[15, 20, 25]</code>.
---	---

 https://youtu.be/mduKvQt_me4 <pre>print("a[5:] = ", a[5:])</pre>	<code>a[5:]</code>: This includes elements from index 5 to the end of the list. So, it includes elements starting from index 5 and goes up to the last element in the list. The result of <code>a[5:]</code> will be a new list containing <code>[30, 35, 40]</code>. This is because it includes all elements in the original list starting from index 5 and going up to the last element. This indicates that the slice of the list starting from index 5 (inclusive) to the end of the list results in the list <code>[30, 35, 40]</code>.
---	--

 https://youtu.be/mduKvQt_me4	<code>a[:3]</code>: This includes elements from the beginning of the list (index 0) up to, but not
---	---

<pre>print("a[:3] = ", a[:3])</pre>	including, index 3. So, it includes elements at index 0, 1, and 2. The result of <code>a[:3]</code> will be a new list containing [5, 10, 15]. This is because it includes the first three elements from the original list <code>a</code>. This indicates that the slice of the list <code>a</code>, starting from the beginning and going up to, but not including, index 3, results in the list [5, 10, 15].
-------------------------------------	---

Loop Statements

https://youtu.be/fQWmZ-U8Ldc <pre>for i in range(10): print(i)</pre>	We have a loop that starts with the word <code>for</code>. It tells the computer to repeat some actions multiple times. Inside the loop, we have a variable called <code>i</code>. Think of <code>i</code> like a little counter that goes up by 1 every time the loop repeats. The loop is set up to run 10 times. We use the <code>range(10)</code> function to create a sequence of numbers from 0 to 9 (10 numbers in total). During each repetition of the loop, the value of <code>i</code> will change to the next number in the sequence (0, 1, 2, 3, ..., 9). The <code>print(i)</code> line inside the loop is responsible for showing the current value of <code>i</code> on the screen. So, the loop starts at 0, prints 0, then goes to 1, prints 1, and so on, until it reaches 9 and prints 9. The loop stops after printing all the numbers from 0 to 9.
---	---

https://youtu.be/fQWmZ-U8Ldc	<code>for</code> loop: The <code>for</code> loop is a way to repeat an action multiple times. It
---	---

```
for i in range(2, 8):  
    print(i)
```

helps us go through a sequence of numbers one by one.

i: This is a variable that the loop uses to keep track of the current number it's working with. In each iteration of the loop, i will hold a different value from the sequence.

range(2, 8): The range() function creates a sequence of numbers. In this case, it starts from 2 and goes up to, but does not include, 8. So, the sequence will be [2, 3, 4, 5, 6, 7].

print(i): Inside the loop, this line prints the current value of i on the screen.

When you run the code, it will execute the loop six times and print the numbers 2 to 7, each on a new line

The loop starts at 2, prints 2, then goes to 3, prints 3, and so on, until it reaches 7 and prints 7. It stops just before 8 because the range does not include 8.



<https://youtu.be/fQWmZ-U8Ldc>

```
for i in range(2, 20, 3):  
    print(i)
```

for loop: The for loop is used to repeat an action multiple times. It helps us go through a sequence of numbers one by one.

i: This is a variable that the loop uses to keep track of the current number it's working with. In each iteration of the loop, i will hold a different value from the sequence.

range(2, 20, 3): The range() function creates a sequence of numbers with a start, stop, and step value. In this case, it starts from 2, goes up to, but does not include, 20, and increments by 3 in each step. So, the sequence will be [2, 5, 8, 11, 14, 17].

	<code>print(i)</code>: Inside the loop, this line prints the current value of <code>i</code> on the screen. When you run the code, it will execute the loop six times and print the numbers 2, 5, 8, 11, 14, and 17, each on a new line The loop starts at 2, prints 2, then goes to 5, prints 5, and so on, until it reaches 17 and prints 17. It increments by 3 in each step. That's how the code works in simple terms! It allows you to print numbers in a specific range with a defined step value using just a few lines of code.
--	--

 https://youtu.be/b_YQqK4j9G8 <pre> i = 1 while i < 6: print(i) i += 1 </pre>	<code>i = 1</code>: This line sets the value of the variable <code>i</code> to 1. Think of <code>i</code> as a counter that we will use to keep track of the current number. while loop: The while loop is a control flow statement that repeats a block of code as long as a certain condition is True. In this case, the loop will continue as long as the value of <code>i</code> is less than 6. <code>i < 6</code>: This is the condition of the while loop. It checks if the value of <code>i</code> is less than 6. If this condition is True, the indented block of code inside the loop will be executed. If it's False, the loop will stop, and the program will move on to the next lines of code. <code>print(i)</code>: Inside the loop, this line prints the current value of <code>i</code> on the screen. <code>i += 1</code>: After printing the value of <code>i</code>, this line increments the value of <code>i</code> by 1. It's like moving to the next number. The loop will continue as long as it is less than 6. So, it will run 5 times because once <code>i</code> becomes 6 (after the fifth iteration), the condition <code>i < 6</code> will be False, and the loop will stop.
--	---



https://youtu.be/b_YQgK4j9G8

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

i = 1: This line sets the value of the variable *i* to 1. Think of *i* as a counter that we will use to keep track of the current number.

while loop: The while loop is a control flow statement that repeats a block of code as long as a certain condition is True. In this case, the loop will continue as long as the value of *i* is less than 6.

i < 6: This is the condition of the while loop. It checks if the value of *i* is less than 6. If this condition is True, the indented block of code inside the loop will be executed. If it's False, the loop will stop, and the program will move on to the next lines of code.

print(i): Inside the loop, this line prints the current value of *i* on the screen.

if i == 3: ... break: This if statement checks if the value of *i* is equal to 3. If *i* is 3, the break statement is executed. The break statement is used to exit the loop immediately, even if the condition of the while loop is still True.

i += 1: After printing the value of *i*, this line increments the value of *i* by 1. It's like moving to the next number.

The loop will continue as long as it is less than 6. However, if *i* becomes 3 during the loop (the value of *i* is 3), the if *i* == 3: ... break block will be executed, and the loop will be terminated immediately.



https://youtu.be/b_YQgK4j9G8

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

i = 0: This line sets the value of the variable *i* to 0. Think of *i* as a counter that we will use to keep track of the current number.

while loop: The while loop is a control flow statement that repeats a block of code as long as a certain condition is True. In this case, the loop will continue as long as the value of *i* is less than 6.

	<code>i < 6</code>: This is the condition of the while loop. It checks if the value of <code>i</code> is less than 6. If this condition is True, the indented block of code inside the loop will be executed. If it's False, the loop will stop, and the program will move on to the next lines of code. <code>i += 1</code>: Inside the loop, this line increments the value of <code>i</code> by 1. It's like moving to the next number. <code>if i == 3: ... continue:</code> This if statement checks if the value of <code>i</code> is equal to 3. If <code>i</code> is 3, the <code>continue</code> statement is executed. The <code>continue</code> statement tells the loop to go back to the beginning of the loop immediately, skipping the rest of the code in the loop for the current iteration. <code>print(i)</code>: Inside the loop, this line prints the current value of <code>i</code> on the screen. The loop will continue as long as it is less than 6. However, if <code>i</code> becomes 3 during the loop (the value of <code>i</code> is 3), the <code>if i == 3: ... continue</code> block will be executed, and the loop will skip the <code>print(i)</code> line for that iteration. It will then go back to the beginning of the loop and continue with the next value of <code>i</code>.
--	--

 https://youtu.be/b_YQqK4j9G8 <pre> i = 1 while i < 6: print(i) i += 1 else: print("i is no longer less than 6") </pre>	<code>i = 1</code>: This line sets the value of the variable <code>i</code> to 1. Think of <code>i</code> as a counter that we will use to keep track of the current number. while loop: The while loop is a control flow statement that repeats a block of code as long as a certain condition is True. In this case, the loop will continue as long as the value of <code>i</code> is less than 6. <code>i < 6</code>: This is the condition of the while loop. It checks if the value of <code>i</code> is less than 6. If this condition is True, the indented block of code inside the loop will be executed. If it's
--	---

	False, the program will move to the else block. print(i): Inside the loop, this line prints the current value of i on the screen. i += 1: After printing the value of i, this line increments the value of i by 1. It's like moving to the next number. The loop will continue as long as i is less than 6. It will print the numbers 1 to 5, each on a new line. else block: In Python, a while loop can have an optional else block. If the loop completes its iterations normally (the condition becomes False), the code inside the else block will be executed. print("i is no longer less than 6"): The line inside the else block prints the message "i is no longer less than 6" after the loop finishes executing.
--	---

Functions

 https://youtu.be/CgwKYRPePXk <pre> def posOrNeg(num): if num > 0: print("Number is positive: ", num) elif num < 0: print("Number is Negative: ", num) else: print("Number is zero: ", num) num = int(input("Enter the number: ")) posOrNeg(num) </pre>	We have defined a function called posOrNeg, which takes a single argument called num. Inside the function, we check the value of num using an if-elif-else statement. If num is greater than 0, we print "The number is positive." If num is less than 0, we print "The number is negative." If num is neither greater than nor less than 0 (i.e., it is equal to 0), we print "The number is zero."
--	---

	Outside the function, we ask the user to input a number using <code>input()</code>. The input is converted to an integer using <code>int()</code> and stored in the variable <code>num</code>. We then call the function <code>posOrNeg(num)</code> and pass the user-input number as an argument to the function. When you run the code, it will ask you to enter a number. Based on the number you enter, it will print whether the number is positive, negative, or zero. That's how the code works in simple terms! It's a function that tells you whether a number is positive, negative, or zero based on your input.
--	---

 https://youtu.be/CgwKYRPePXk Maximum of input 2 numbers <pre>def max(num1, num2): if num1 > num2: return num1 else: return num2 a = int(input("Enter first number: ")) b = int(input("Enter second number: ")) print("max number is :", max(a, b))</pre>	<code>def max(num1, num2):</code>: This line creates a function named <code>max</code> that takes two numbers, <code>num1</code> and <code>num2</code>, as inputs. <code>if num1 > num2:</code>: This checks if <code>num1</code> is greater than <code>num2</code>. <code>return num1:</code> If <code>num1</code> is greater, it immediately returns <code>num1</code> as the maximum. <code>else:</code>: If <code>num1</code> is not greater (i.e., it's less than or equal to <code>num2</code>), the code inside the <code>else</code> block runs. <code>return num2:</code> This returns <code>num2</code> as the maximum. <code>a = int(input("Enter first number: "))</code>: The user is asked to input the first number, which is stored in the variable <code>a</code>.
---	--

	<code>b = int(input("Enter second number: "))</code>: The user is asked to input the second number, which is stored in the variable <code>b</code>. <code>print("max number is:", max(a, b))</code>: The <code>max</code> function is called with <code>a</code> and <code>b</code> as inputs. The function compares the numbers and returns the greater one, which is then printed as the maximum number. When you run the code, it will ask you to enter two numbers. After entering the numbers, it will tell you which one is the greater (maximum) number among the two.
--	--

 https://youtu.be/mdSvNmsb6f8 Addition of two numbers <pre>def sum(num1, num2): return num1 + num2 a = int(input("Enter first number: ")) b = int(input("Enter second number: ")) c = sum(a, b) print("answer is : ", c)</pre>	We defined a function called <code>sum</code> that takes two arguments, <code>num1</code> and <code>num2</code>. Inside the function, we add <code>num1</code> and <code>num2</code> together using the <code>+</code> operator and use the <code>return</code> statement to send the result back as the output of the function. Outside the function, we ask the user to input two numbers using <code>input()</code>. The inputs are converted to integers using <code>int()</code> and stored in the variables <code>a</code> and <code>b</code>. We then call the <code>sum</code> function and pass the values of <code>a</code> and <code>b</code> as arguments. The function returns the sum of <code>a</code> and <code>b</code>, and we store it in the variable <code>c</code>. Finally, we print the value of <code>c</code> along with a message saying "The sum is:". When you run the code, it will ask you to enter two numbers. After you provide the inputs, it will calculate their sum and display the result on the screen. That's how the code works in simple terms! It's a function that takes two numbers, adds them together, and then shows you the result.
---	--

Geometric Functions



<https://youtu.be/UL2MXmrrKj8>

Function calculating area of circle

pi = 3.14

```
def areaOfCircle(radius):  
    return 2 * pi * radius
```

```
radius = int(input("Enter radius of circle: "))  
print("Area of circle= ",  
      areaOfCircle(radius) )
```

We first set a variable pi to the value 3.14. This is a common approximation of the mathematical constant π (pi), which is used in calculations involving circles.

We define a function called areaOfCircle that takes one argument, radius.

Inside the function, we use the formula for the circumference of a circle: $2 * \pi * \text{radius}$. This calculates the circumference of the circle based on the given radius.

We use the return statement to send the calculated circumference as the output of the function.

Outside the function, we ask the user to input the radius of the circle using input(). The input is converted to an integer using int() and stored in the variable radius.

We then call the function areaOfCircle and pass the value of radius as an argument. The function returns the calculated circumference, and we store it in the variable c.

Finally, we print the value of c along with a message saying "Area of the circle =".

When you run the code, it will ask you to enter the radius of the circle. After you provide the input, it will calculate the circumference of the circle using the formula $2 * \pi * \text{radius}$ and display the result on the screen.



<https://youtu.be/UL2MXmrrKj8>

Function calculating area of square

We define a function called areaOfSquare that takes one argument, side.

```
def areaOfSquare(side):  
    return side ** 2
```

```
side = int(input("Enter side of square: "))  
print("Area of square=",  
      areaOfSquare(side))
```

Inside the function, we calculate the area of the square using the formula: $\text{side} ** 2$. This formula multiplies the length of the side by itself to find the area.

We use the return statement to send the calculated area as the output of the function.

Outside the function, we ask the user to input the length of the side of the square using `input()`. The input is converted to an integer using `int()` and stored in the variable `side`.

We then call the function `areaOfSquare` and pass the value of `side` as an argument. The function returns the calculated area, and we store it in the variable `area`.

Finally, we print the value of `area` along with a message saying "Area of the square =".

When you run the code, it will ask you to enter the length of the side of the square. After you provide the input, it will calculate the area of the square using the formula $\text{side} ** 2$ and display the result on the screen.



<https://youtu.be/QUecAQ4VwF8>

Write a program to input marks of five subjects Physics, Chemistry, Biology, Mathematics and Computer. Calculate percentage and grade according to following:

Percentage $\geq 90\%$: Grade A
Percentage $\geq 80\%$: Grade B
Percentage $\geq 70\%$: Grade C
Percentage $\geq 60\%$: Grade D
Percentage $\geq 40\%$: Grade E
Percentage $< 40\%$: Grade F

```
physics = int(input("Enter Physics marks: "))  
chemistry = int(input("Enter chemistry marks: "))  
biology = int(input("Enter biology marks: "))
```

```

mathematics = int(input("Enter mathematics marks: "))
computer = int(input("Enter computer marks: "))

total = physics + chemistry + biology + mathematics + computer
percentage =( total / 500) * 100

def grade(percentage):
    if percentage >= (90/100)*100:
        return("Grade A")
    elif percentage >= (80/100)*100:
        return("Grade B")
    elif percentage >= (70/100)*100:
        return("Grade C")
    elif percentage >= (60/100)*100:
        return("Grade D")
    elif percentage >= (40/100)*100:
        return("Grade E")
    else:
        return("Grade F")

print(percentage)
print("Grade is:", grade(percentage))

```

We ask the user to input marks in five subjects: Physics, Chemistry, Biology, Mathematics, and Computer.

We calculate the total marks by adding the marks in all five subjects.

We calculate the percentage by dividing the total marks by 500 and then multiplying by 100 to get the percentage value.

We define a function called grade that takes the percentage as an argument.

Inside the function, we use an if-elif-else statement to determine the grade based on the percentage.

If the percentage is greater than or equal to 90, it returns "Grade A".

If the percentage is between 80 and 89 (inclusive), it returns "Grade B".

If the percentage is between 70 and 79 (inclusive), it returns "Grade C".

If the percentage is between 60 and 69 (inclusive), it returns "Grade D".

If the percentage is between 40 and 59 (inclusive), it returns "Grade E".

If the percentage is less than 40, it returns "Grade F".

Outside the function, we print the percentage and the result of the grade(percentage) function, which will show the corresponding grade.

When you run the code, it will ask you to enter marks in each subject, calculate the percentage, and then display the percentage and the corresponding grade based on the percentage.



<https://youtu.be/XuxkGVhLpFU>

Enter length in centimeter and convert it into meter and kilometer.

```
centimeter = float(input("Enter length in centimeter: "))
meter = 0
kilometer = 0

meter = centimeter / 100
kilometer = centimeter / 100000

print(centimeter,"= ", meter, " meter")
print(centimeter,"= ", kilometer, " kilometer")
```

We ask the user to input a length in centimeters using input(). The input is converted to a floating-point number using float() and stored in the variable centimeter.

We calculate the length in meters by dividing centimeter by 100. This is because there are 100 centimeters in 1 meter. The result is stored in the variable meter.

We calculate the length in kilometers by dividing centimeter by 100000. This is because there are 100,000 centimeters in 1 kilometer. The result is stored in the variable kilometer.

We use the print() function to display the original length in centimeters along with the converted lengths in meters and kilometers.

When you run the code, it will ask you to enter a length in centimeters. After you provide the input, it will convert the length to meters and kilometers and display the results on the screen.



<https://youtu.be/Fs28059hANQ>

Enter two angles of a triangle and find the third angle.

```
angle1 = int(input("Enter first angle: "))
angle2 = int(input("Enter second angle: "))
angle3 = None

angle3 = 180 - (angle1 + angle2)
```

```
print("Third angle of a triangle is:", angle3)
```

We ask the user to input the measures of the first and second angles of a triangle using `input()`. The inputs are converted to integers using `int()` and stored in the variables `angle1` and `angle2`.

We initialise the variable `angle3` as `None`. This is just a placeholder value that will be updated later.

We calculate the measure of the third angle of the triangle by subtracting the sum of `angle1` and `angle2` from 180. In a triangle, the sum of the measures of all three angles is always 180 degrees.

We assign the calculated value to the variable `angle3`.

Finally, we use the `print()` function to display the value of `angle3` along with a message saying "Third angle of a triangle is:".

When you run the code, it will ask you to enter the measures of the first and second angles of a triangle. After you provide the inputs, it will calculate the measure of the third angle by subtracting the sum of the first two angles from 180. It will then display the result on the screen.



<https://youtu.be/Fs28059hANQ>

Write a program to check whether the triangle is equilateral, isosceles or scalene triangle.

```
side1 = int(input('Enter first side of a Triangle: '))  
side2 = int(input('Enter second side of a Triangle: '))  
side3 = int(input('Enter third side of a Triangle:'))
```

```
if (side1 == side2 == side3):  
    print("triangle is an equilateral triangle")  
elif (side1 == side2 != side3):  
    print("triangle is an isosceles triangle")  
elif (side2 == side3 != side1):  
    print("triangle is an isosceles triangle")  
elif (side3 == side1 != side2):  
    print("triangle is an isosceles triangle")  
else:  
    print("triangle is a scalene triangle")
```

We ask the user to input the lengths of the three sides of a triangle using `input()`. The inputs are converted to integers using `int()` and stored in the variables `side1`, `side2`, and `side3`.

We use if-elif-else statements to check the conditions and classify the type of triangle based on the side lengths.

If all three sides are equal (i.e., `side1 == side2 == side3`), the triangle is classified as an equilateral triangle. An equilateral triangle has all three sides of equal length.

If two sides are equal and the third side is different, the triangle is classified as an isosceles triangle. An isosceles triangle has two sides of equal length.

We use the logical operator `or` to check the conditions (`side1 == side2 != side3`), (`side2 == side3 != side1`), and (`side3 == side1 != side2`). If any of these conditions is true, the triangle is classified as an isosceles triangle.

If none of the above conditions are met, the triangle is classified as a scalene triangle. A scalene triangle has all three sides of different lengths.

We use the `print()` function to display the classification of the triangle based on the conditions met.

When you run the code, it will ask you to enter the lengths of the three sides of a triangle. After you provide the inputs, it will classify the triangle as equilateral, isosceles, or scalene based on the side lengths. It will then display the classification on the screen.



<https://youtu.be/eAZ0LyyFnOY>

Write a program to get the cost and selling price and calculate profit or loss.

```
cp = int(input("Enter the Cost Price: "))
sp = int(input("Enter the Selling Price: "))
```

```
if (sp > cp):
    print("There is a profit of", sp - cp)
else:
    print("There is a loss of", cp - sp)
```

The program prompts the user to enter the cost price and selling price of the product. It stores the cost price in the variable `cp` and the selling price in the variable `sp`. The program then checks if the selling price is greater than the cost price by using the condition `sp > cp`. If the condition is true (i.e., the selling price is greater than the cost price), it means there is a profit. The program calculates the profit by subtracting the cost price from the selling price (`sp - cp`). The program then prints the message "There is a profit of" followed by the calculated profit.

If the condition is false (i.e., the selling price is less than or equal to the cost price), it means there is a loss. The program calculates the loss by subtracting the selling price from the cost price ($cp - sp$).

The program then prints the message "There is a loss of" followed by the calculated loss.



<https://youtu.be/eAZ0LyyFnOY>

Write a program to print all natural numbers from 1 to n. - using while loop

```
number = int(input("Please Enter any Number: "))
```

```
i = 1
```

```
print("The List of Natural Numbers from 1 to {0} are".format(number))
```

```
while ( i <= number):
```

```
    print (i, end = ' ')
```

```
    i = i + 1
```

The program prompts the user to enter any number, which is stored in the variable number.

It initializes a variable i to 1. This variable will be used to iterate through and print the natural numbers.

The program prints a message indicating that it will display the list of natural numbers from 1 to the input number.

It enters a while loop that will execute as long as i is less than or equal to the input number (number).

Inside the loop, it prints the value of i followed by two spaces (for formatting) using the print function. The end = ' ' parameter ensures that the numbers are printed horizontally on the same line.

After printing the current value of i, it increments i by 1 using $i = i + 1$, which prepares the next value to be printed in the next iteration of the loop.

The loop continues until it becomes greater than the input number. Once the condition $i \leq \text{number}$ is false, the loop terminates.



<https://youtu.be/RZdjvdHA6aQ>

Write a program to print all even numbers between 1 to 100. - using while loop

```
max = 100
```

```
num = 1
```

```
while num <= max:
```

```
    if(num % 2 == 0):
```

```
print("{0}".format(num))  
num = num + 1
```

The program sets the value of the variable max to 100. This is the upper limit of the range of numbers that will be checked.

It initialises a variable num to 1. This variable will be used to iterate through the numbers from 1 to max.

The program enters a while loop that will execute as long as num is less than or equal to max.

Inside the loop, there is an if statement that checks if the current value of num is even. It does this by checking if num is divisible by 2 without leaving any remainder (num % 2 == 0).

If the condition in the if statement is true (i.e., num is even), it prints the value of num using the print function.

After printing the current value of num, it increments num by 1 using num = num + 1, which prepares the next number to be checked in the next iteration of the loop.

The loop continues until num becomes greater than max. Once num exceeds 100, the loop terminates.



https://youtu.be/bQws_dpCzg0

Write a program to print a multiplication table of any number.

```
number = int(input("Enter the number of which the user wants to print the multiplication  
table: "))
```

```
print ("The Multiplication Table of: ", number)  
for count in range(1, 11):  
    print (number, 'x', count, '=', number * count)
```

The program prompts the user to enter a number for which they want to generate the multiplication table. The entered number is stored in the variable number.

It then prints a message indicating that it will display the multiplication table of the entered number.

The program enters a for loop that will iterate through the range of numbers from 1 to 10 (exclusive). The loop variable count takes each value from 1 to 10 in sequence.

Inside the loop, it calculates the product of the entered number and the current value of count (i.e., number * count), which gives one entry in the multiplication table.

The program then prints the current entry in the multiplication table in the format: number x count = result.

The loop will continue for each value of count from 1 to 10, and the multiplication table will be displayed for the entered number.



https://youtu.be/bQws_dpCzg0

Write a program to calculate the sum of digits of a number.

```
def getSum(n):
```

```
    sum = 0
```

```
    for digit in str(n):
```

```
        sum = sum + int(digit)
```

```
    return sum
```

```
n = int(input("Enter any number"))
```

```
print(getSum(n))
```

The function getSum(n) takes an integer n as input.

It initializes a variable sum to 0. This variable will be used to store the sum of the digits.

The function converts the input number n into a string using str(n).

It then enters a for loop that iterates over each character (digit) in the string representation of the number.

Inside the loop, it converts each digit back to an integer using int(digit) and adds it to the current value of sum.

The loop continues until all digits in the number have been processed and their values added to sum.

Finally, the function returns the calculated sum.

Outside the function:

The program prompts the user to enter any number, which is stored in the variable n.

It calls the getSum(n) function with the input number n and prints the returned sum.

Turtle Graphics



<https://youtu.be/cLLdroNnQkU>

```
import turtle
```

```
pen = turtle.Turtle()
```

```
pen.forward(100) # This makes our first forward (pointing north)
```

```
pen.left(90) # Then the turtle turns left 90 degrees
```

```
pen.forward(100) # This makes our second forward (pointing west)
```

```
pen.left(90) # Then the turtle turns left 90 degrees
```

```
pen.forward(100) # This makes our third forward (pointing south)
pen.left(90) # Then the turtle turns left 90 degrees
pen.forward(100) # This makes our fourth forward (pointing east)
```

import turtle: The code imports the Turtle module, which allows us to create a turtle object for drawing.

pen = turtle.Turtle(): A turtle object named pen is created. This turtle will be used to draw on the screen.

pen.forward(100): The turtle moves forward by 100 units (pixels by default). This creates the first side of the square, pointing north.

pen.left(90): The turtle turns left by 90 degrees. This means the turtle now faces west.

pen.forward(100): The turtle moves forward by 100 units again. This creates the second side of the square, pointing west.

pen.left(90): The turtle turns left by 90 degrees again. This means the turtle now faces south.

pen.forward(100): The turtle moves forward by 100 units again. This creates the third side of the square, pointing south.

pen.left(90): The turtle turns left by 90 degrees again. This means the turtle now faces east.

pen.forward(100): The turtle moves forward by 100 units again. This creates the fourth side of the square, pointing east.



https://youtu.be/gTq_VToUvcA

```
import turtle
pen = turtle.Turtle()
for x in range(4):
    pen.forward(100)
    pen.left(90)
```

import turtle: The code imports the Turtle module, which allows us to create a turtle object for drawing.

pen = turtle.Turtle(): A turtle object named pen is created. This turtle will be used to draw on the screen.

for x in range(4):: The code enters a for loop that will iterate four times. The variable x takes the values 0, 1, 2, and 3 during each iteration.

pen.forward(100): Inside the loop, the turtle moves forward by 100 units (pixels by

	default). This creates one side of the square during each iteration. <code>pen.left(90)</code>: After moving forward, the turtle turns left by 90 degrees. This means the turtle changes its direction to create a right-angle turn. The loop continues to execute four times, and during each iteration, the turtle moves forward and turns left by 90 degrees, creating all four sides of the square. Now, if you run this code, it will draw a square on the screen using the turtle, just like the previous example. The for loop makes the code more concise and avoids repetitive lines.
--	--

 https://youtu.be/Rnzorb9akyc <pre> import turtle pen = turtle.Turtle() for x in range (100): pen.forward(x * 4) pen.left(90) </pre>	<code>import turtle</code>: The code imports the Turtle module, which allows us to create a turtle object for drawing. <code>pen = turtle.Turtle()</code>: A turtle object named pen is created. This turtle will be used to draw on the screen. <code>for x in range(100):</code>: The code enters a for loop that will iterate 100 times. The variable x takes the values from 0 to 99 during each iteration. <code>pen.forward(x * 4)</code>: Inside the loop, the turtle moves forward by a distance equal to x * 4 units (pixels by default). As x increases from 0 to 99, the distance moved by the turtle also increases, creating squares of different sizes. <code>pen.left(90)</code>: After moving forward, the turtle turns left by 90 degrees. This means the turtle changes its direction to create a right-angle turn after drawing each square. The loop continues to execute 100 times, and during each iteration, the turtle moves
---	--

	forward a distance determined by $x * 4$ and then turn left by 90 degrees. This results in a series of connected squares with increasing sizes. When you run this code, you will see a visually interesting pattern of connected squares on the screen. The first square will be very small, and with each iteration, the square's size will increase as x increases.
--	---

 https://youtu.be/_dg9ILJWnsQ <pre> import turtle pen = turtle.Turtle() pen.pencolor("red") for x in range (100): pen.forward(x * 4) pen.left(90) </pre>	import turtle: The code imports the Turtle module, which allows us to create a turtle object for drawing. pen = turtle.Turtle(): A turtle object named pen is created. This turtle will be used to draw on the screen. pen.pencolor("red"): The pen color is set to "red." This means that the turtle will draw the squares using a red pen. for x in range(100):: The code enters a for loop that will iterate 100 times. The variable x takes the values from 0 to 99 during each iteration. pen.forward($x * 4$): Inside the loop, the turtle moves forward by a distance equal to $x * 4$ units (pixels by default). As x increases from 0 to 99, the distance moved by the turtle also increases, creating squares of different sizes. pen.left(90): After moving forward, the turtle turns left by 90 degrees. This means the turtle changes its direction to create a right-angle turn after drawing each square.
---	--

	The loop continues to execute 100 times, and during each iteration, the turtle moves forward a distance determined by $x * 4$ and then turns left by 90 degrees. This results in a series of connected squares with increasing sizes, all drawn in red color. When you run this code, you will see a visually interesting pattern of connected squares in red color on the screen. The first square will be very small, and with each iteration, the square's size will increase as x increases. The squares will be drawn in red due to the <code>pen.pencolor("red")</code> instruction at the beginning of the code.
--	---

 https://youtu.be/XZyQdOPFJZU <pre> import turtle pen = turtle.Turtle() colors = ["red", "yellow", "blue", "green"] for x in range(100): pen.pencolor(colors[x % 4]) pen.forward(x * 4) pen.left(90) </pre>	<code>import turtle</code>: The code imports the Turtle module, which allows us to create a turtle object for drawing. <code>pen = turtle.Turtle()</code>: A turtle object named <code>pen</code> is created. This turtle will be used to draw on the screen. <code>colors = ["red", "yellow", "blue", "green"]</code>: A list named <code>colors</code> is created, containing four colors: "red," "yellow," "blue," and "green." <code>for x in range(100):</code>: The code enters a for loop that will iterate 100 times. The variable <code>x</code> takes the values from 0 to 99 during each iteration. <code>pen.pencolor(colors[x % 4])</code>: Inside the loop, the turtle's pen color is set to one of the colors in the <code>colors</code> list based on the remainder of <code>x</code> when divided by 4. This means that the pen color will cycle through "red," "yellow," "blue," and "green" repeatedly as <code>x</code> increases from 0 to 99. <code>pen.forward(x * 4)</code>: After setting the pen color, the turtle moves forward by a distance equal to $x * 4$ units (pixels by default). As <code>x</code> increases from 0 to 99, the distance moved
--	--

	by the turtle also increases, creating squares of different sizes. <code>pen.left(90)</code>: After moving forward, the turtle turns left by 90 degrees. This means the turtle changes its direction to create a right-angle turn after drawing each square. The loop continues to execute 100 times, and during each iteration, the turtle sets its pen color from the colors list, moves forward a distance determined by $x * 4$, and then turns left by 90 degrees. This results in a series of connected squares with increasing sizes, and their colors cycle through "red," "yellow," "blue," and "green." When you run this code, you will see a visually interesting pattern of connected squares, each with a different color. The first square will be very small and red, and with each iteration, the square's size will increase, and the pen color will change according to the cycle of colors in the colors list.
--	---

 https://youtu.be/vRIKb5jGs-o <pre> import turtle pen = turtle.Turtle() colors = ["red", "yellow", "blue", "green"] for x in range(100): pen.pencolor(colors[x % 4]) pen.forward(x * 4) pen.left(90) #pen.left(60) #pen.left(120) </pre>	<code>import turtle</code>: The code imports the Turtle module, which allows us to create a turtle object for drawing. <code>pen = turtle.Turtle()</code>: A turtle object named pen is created. This turtle will be used to draw on the screen. <code>colors = ["red", "yellow", "blue", "green"]</code>: A list named colors is created, containing four colors: "red," "yellow," "blue," and "green." <code>for x in range(100)::</code> The code enters a for loop that will iterate 100 times. The variable x takes the values from 0 to 99 during each iteration. <code>pen.pencolor(colors[x % 4])</code>: Inside the loop, the turtle's pen color is set to one of the colors in the colors list based on the
---	--

	remainder of x when divided by 4. This means that the pen color will cycle through "red," "yellow," "blue," and "green" repeatedly as x increases from 0 to 99. <code>pen.forward(x * 4)</code>: After setting the pen color, the turtle moves forward by a distance equal to $x * 4$ units (pixels by default). As x increases from 0 to 99, the distance moved by the turtle also increases, creating squares of different sizes. <code>pen.left(90)</code>: After moving forward, the turtle turns left by 90 degrees. This means the turtle changes its direction to create a right-angle turn after drawing each square. The loop continues to execute 100 times, and during each iteration, the turtle sets its pen color from the colors list, moves forward a distance determined by $x * 4$, and then turns left by 90 degrees. This results in a series of connected squares with increasing sizes, and their colors cycle through "red," "yellow," "blue," and "green." The lines <code>pen.left(60)</code> and <code>pen.left(120)</code> are currently commented out by using the <code>#</code> symbol at the beginning of each line. When a line is commented out, it means that line of code is not executed during the program's execution. In this case, the two lines have no effect on the drawing because they are not active. When you run this code, you will see a visually interesting pattern of connected squares, each with a different color. The first square will be very small and red, and with each iteration, the square's size will increase, and the pen color will change according to the cycle of colors in the colors list. The lines <code>pen.left(60)</code> and <code>pen.left(120)</code> are not affecting the drawing because they are commented out.
--	--



<https://youtu.be/9nYvu2BA4Jc>

```
import turtle
turtle.Screen().bgcolor("black")

pen = turtle.Turtle()
colors = ["red", "yellow", "blue", "green"]
for x in range(100):
    pen.pencolor(colors[x % 4])
    pen.circle(x * 10)
    pen.left(91)
```

import turtle: The code imports the Turtle module, which allows us to create a turtle object for drawing.

turtle.Screen().bgcolor("black"): The background color of the drawing area (screen) is set to black.

pen = turtle.Turtle(): A turtle object named pen is created. This turtle will be used to draw on the screen.

colors = ["red", "yellow", "blue", "green"]: A list named colors is created, containing four colors: "red," "yellow," "blue," and "green."

for x in range(100):: The code enters a for loop that will iterate 100 times. The variable x takes the values from 0 to 99 during each iteration.

pen.pencolor(colors[x % 4]): Inside the loop, the turtle's pen color is set to one of the colors in the colors list based on the remainder of x when divided by 4. This means that the pen color will cycle through "red," "yellow," "blue," and "green" repeatedly as x increases from 0 to 99.

pen.circle(x * 10): After setting the pen color, the turtle draws a circle with a radius equal to x * 10 units (pixels by default). As x increases from 0 to 99, the size of the circle also increases.

pen.left(91): After drawing the circle, the turtle turns left by 91 degrees. This angle is slightly more than 90 degrees, which makes the circles overlap and form an interesting pattern.

The loop continues to execute 100 times, and during each iteration, the turtle sets its pen color from the colors list, draws a circle with a radius determined by x * 10, and then turns left by 91 degrees. This creates a series of connected circles, each with a

	different color, and they overlap to form an intriguing design. When you run this code, you will see a visually captivating pattern of connected circles on a black background. The circles will be drawn in different colors (red, yellow, blue, and green) and overlap to create a mesmerising effect.
--	--

 https://youtu.be/pA08PL6vT6g <pre> import turtle pen = turtle.Turtle() colors = ["red", "yellow", "blue", "green", "purple", "orange"] for x in range(100): pen.pencolor(colors[x % len(colors)]) pen.forward(x * 10) pen.left(120) </pre>	import turtle: The code imports the Turtle module, which allows us to create a turtle object for drawing. pen = turtle.Turtle(): A turtle object named pen is created. This turtle will be used to draw on the screen. colors = ["red", "yellow", "blue", "green", "purple", "orange"]: A list named colors is created, containing six colors: "red," "yellow," "blue," "green," "purple," and "orange." for x in range(100):: The code enters a for loop that will iterate 100 times. The variable x takes the values from 0 to 99 during each iteration. pen.pencolor(colors[x % len(colors)]): Inside the loop, the turtle's pen color is set to one of the colors in the colors list based on the remainder of x when divided by the length of the colors list. This ensures that the pen color will cycle through the colors in the list repeatedly as x increases from 0 to 99. pen.forward(x * 10): After setting the pen color, the turtle moves forward by a distance equal to x * 10 units (pixels by default). As x increases from 0 to 99, the distance moved by the turtle also increases, creating triangles of different sizes. pen.left(120): After moving forward, the turtle turns left by 120 degrees. This angle
--	---

	creates an equilateral triangle (all sides are of equal length). The loop continues to execute 100 times, and during each iteration, the turtle sets its pen color from the colors list, moves forward a distance determined by $x * 10$, and then turns left by 120 degrees. This results in a series of connected triangles with increasing sizes and changing colors as they cycle through the colors in the colors list. When you run this code, you will see a visually captivating pattern of connected triangles. The triangles will be drawn in different colors (red, yellow, blue, green, purple, and orange) and overlap to form an intriguing design.
--	---

 https://youtu.be/ZT1ZelKw0ic <pre> import turtle pen = turtle.Turtle() # You can choose between 2 and 6 sides for some cool shapes! sides = int(input("Enter the number of sides")) colors = ["red", "yellow", "blue", "orange", "green", "purple", "skyblue", "brown", "pink"] for x in range(360): pen.pencolor(colors[x % sides]) pen.forward(x * 2 + x) pen.left(360 / sides) pen.width(x * sides / 100) </pre>	import turtle: The code imports the Turtle module, which allows us to create a turtle object for drawing. pen = turtle.Turtle(): A turtle object named pen is created. This turtle will be used to draw on the screen. sides = int(input("Enter the number of sides")): The user is prompted to enter the number of sides for the shape. The input is stored in the variable sides, and it must be an integer between 2 and 6. colors = ["red", "yellow", "blue", "orange", "green", "purple", "skyblue", "brown", "pink"]: A list named colors is created, containing several color names. for x in range(360):: The code enters a for loop that will iterate 360 times. The variable x takes the values from 0 to 359 during each iteration. pen.pencolor(colors[x % sides]): Inside the loop, the turtle's pen color is set to one of
--	---

	the colors in the colors list based on the remainder of x when divided by the number of sides. This ensures that the pen color will cycle through the colors in the list repeatedly as x increases from 0 to 359. <code>pen.forward($x * 2 + x$)</code>: After setting the pen color, the turtle moves forward by a distance equal to $(x * 2 + x)$ units (pixels by default). As x increases from 0 to 359, the distance moved by the turtle increases, creating the shape with changing length sides. <code>pen.left($360/\text{sides}$)</code>: After moving forward, the turtle turns left by an angle of $360/\text{sides}$ degrees. This angle is determined by the number of sides in the shape and ensures that the turtle will create a closed shape. <code>pen.width($x * \text{sides} / 100$)</code>: The width of the turtle's pen is set to $(x * \text{sides} / 100)$. As x increases, the width of the pen also increases, creating varying line thicknesses as the shape is drawn. The loop continues to execute 360 times, and during each iteration, the turtle sets its pen color, moves forward a distance determined by $(x * 2 + x)$, turns left by $360/\text{sides}$ degrees, and changes the pen width. This results in a dynamic drawing of a shape with the specified number of sides. When you run this code, you will be prompted to enter the number of sides for the shape. Based on your input, the turtle will draw a shape with that number of sides using different colors and varying line thicknesses, resulting in a visually interesting and colorful design.
--	--